Proyecto Final: Predicción del precio de la gasolina en Guatemala

Descripción general

Cada equipo construirá un pipeline de datos de punta a punta para monitorear el precio de los combustibles en Guatemala y predecir si subirá o bajará la semana siguiente. La fuente principal son fotos de precios que comparte el catedrático: el pipeline las guarda en AWS S3, las convierte en datos con Databricks usando la arquitectura medallón (Bronce → Plata → Oro) y publica precios y predicciones en una API REST en un Droplet de DigitalOcean.

Contexto Guatemala: el Ministerio de Energía y Minas (MEM) publica cada martes precios de referencia de gasolina súper, regular y diésel, en quetzales por galón, vigentes de martes a lunes. Los precios cambian por departamento y por modalidad (autoservicio o servicio completo), siguen al precio internacional del petróleo y, en algunos periodos, se ven afectados por subsidios temporales del Gobierno.

Al terminar el proyecto, el equipo debe poder demostrar que:

1. Conecta Databricks a buckets de S3 mediante *storage credentials* y *external locations* de Unity Catalog.
2. Extrae precios estructurados a partir de fotos (OCR o un modelo de visión) y los valida con reglas de calidad.
3. Implementa las tres capas de la arquitectura medallón con tablas Delta.
4. Entrena un modelo que predice si el precio de cada combustible sube o baja la semana siguiente, y lo evalúa contra los precios reales.
5. Usa correctamente los clusters de Databricks: un cluster interactivo para desarrollar y un job cluster para ejecutar el pipeline de forma automatizada.
6. Despliega una API sobre la capa Oro en un Droplet, accediendo por SSH y aislando dependencias con Docker, conda, uv o venv.
7. Documenta y presenta su solución de forma que otra persona pueda reproducirla.

Arquitectura de referencia

Todos los equipos implementan la misma arquitectura; todos reciben las mismas fotos y lo que cambia es cómo extraen los precios y qué modelo eligen.



arquitectura medallón · S3, Databricks y Droplet

Cada tarea del Job lee y escribe en el bucket de su capa mediante una external location. La API del Droplet lee solo la capa Oro: el usuario final la consulta por HTTP y el equipo la despliega por SSH.

Infraestructura y accesos por equipo

El curso se organiza en 7 equipos. Al inicio del proyecto cada equipo recibe los accesos de esta tabla; `XX` es el número del equipo con dos dígitos (01 a 07).

Recurso	Qué recibe el equipo	Convención de nombre
Número de equipo	Identificador único	`equipo-XX`
Llaves SSH	Llave SSH 1 y Llave SSH 2 para entrar al Droplet	Una llave por integrante responsable del despliegue
Droplet DigitalOcean	IP, usuario Linux propio y puerto asignado	Usuario `equipo-XX`, puerto `80XX`
Bucket Bronce (S3)	Lectura de la carpeta compartida de fotos y escritura en su prefijo	`s3://<bucket-bronce>/equipo-XX/`
Bucket Plata (S3)	Prefijo para tablas limpias	`s3://<bucket-plata>/equipo-XX/`
Bucket Oro (S3)	Prefijo para tablas de negocio	`s3://<bucket-oro>/equipo-XX/`

Recurso	Qué recibe el equipo	Convención de nombre
Workspace Databricks	Acceso al workspace, a un cluster compartido y permiso para crear jobs	Catálogo o esquemas con prefijo `equipo_XX`

Los Droplets pueden ser compartidos entre varios equipos. Por eso cada equipo trabaja solo con su usuario, su carpeta y su puerto; tocar recursos de otro equipo se considera falta grave.

Fase 1: Capa Bronce (ingesta de fotos)

La capa Bronce guarda las fotos tal como llegan, junto con sus metadatos, para poder reprocesarlas si cambia la lógica de extracción.

1. El catedrático sube fotos de precios a `s3://<bucket-bronce>/fotos/AAAA-MM-DD/` (por ejemplo, rótulos de gasolineras o tablas de precios de referencia del MEM). Esta carpeta es de solo lectura para los equipos.
2. Leer las fotos con Spark usando el formato `binaryFile` (o Auto Loader con `cloudFiles.format = binaryFile`) a través de la external location del bucket Bronce.
3. Escribir la tabla Delta `bronze.fotos_precios` en el prefijo del equipo, con el contenido binario, `path`, `modificationTime`, `length` y las columnas de auditoría:
   - `_ingestion_timestamp`: fecha y hora de la carga.
   - `_source_file`: ruta de la foto de origen.
4. Cargar de forma incremental: cuando el catedrático comparta fotos nuevas, solo se procesan las que no se habían cargado.
5. Opcional, para tener más historia para el modelo: cargar en Bronce una fuente complementaria en su formato original, como el histórico de precios de referencia del MEM o el precio internacional del petróleo.

Criterio de aceptación: el número de filas en `bronze.fotos_precios` coincide con el número de fotos en la carpeta, y una segunda carga no duplica fotos.

Fase 2: Capa Plata (extracción y limpieza de precios)

La capa Plata convierte cada foto en registros de precio limpios: una fila por fecha, ubicación, combustible y modalidad.

1. Extraer el texto de cada foto con OCR (por ejemplo, Tesseract o EasyOCR) o con un modelo de visión. La librería se instala en el cluster (librería del cluster o `%pip`).

2. Convertir el texto en la tabla `silver.precios` con estas columnas mínimas:
   - `fecha` (`DATE`, tomada de la carpeta o del metadato de la foto).
   - `combustible`: `super`, `regular` o `diesel`.
   - `precio_q_galon`: decimal en quetzales por galón.
   - `modalidad`: autoservicio o servicio completo, si aparece.
   - `ubicacion`: estación, municipio o departamento, si aparece.
   - `foto_origen`: ruta de la foto en Bronce.
3. Implementar al menos 3 reglas de calidad, por ejemplo: precio dentro de un rango definido y justificado por el equipo, combustible dentro del catálogo, fecha no futura. Las fotos o filas que fallen van a `silver.precios_rejected` con el motivo del rechazo.
4. Deduplicar con la llave `fecha + ubicacion + combustible + modalidad` y hacer la carga idempotente con `MERGE`.
5. Medir la calidad de la extracción: revisar a mano al menos 20 fotos y reportar el porcentaje de precios extraídos correctamente.

Criterio de aceptación: el notebook muestra cuántas fotos se procesaron, cuántas se rechazaron y por qué, y la exactitud de la revisión manual; una segunda ejecución produce el mismo resultado.

Fase 3: Capa Oro (indicadores y modelo de predicción)

La capa Oro responde la pregunta central del proyecto: ¿el precio de cada combustible subirá o bajará la semana siguiente? Es la única capa que consume la API.

1. `gold.precio_semanal`: precio promedio, mínimo y máximo por semana y combustible. La semana va de martes a lunes, igual que los precios de referencia del MEM.
2. `gold.variacion_semanal`: diferencia en quetzales y en porcentaje contra la semana anterior, y la etiqueta `direccion` (`sube`, `baja` o `se_mantiene`) con un umbral definido por el equipo, por ejemplo ±Q0.10.
3. `gold.features_modelo`: variables para predecir, por ejemplo variación de las últimas 1 a 4 semanas, promedio móvil, diferencia entre súper y diésel y, si se cargó, el precio internacional del petróleo.
4. Entrenar un modelo de clasificación (regresión logística, árbol de decisión o similar) que prediga la `direccion` de la semana siguiente para cada combustible. Compararlo contra una línea base simple: "se repite la dirección de la semana anterior".
5. Registrar los experimentos con MLflow en Databricks: parámetros, métricas y el modelo elegido.

6. `gold.predicciones`: una fila por combustible y semana con la predicción, su probabilidad y, cuando se conozca, la dirección real.
7. Agregar comentarios a cada tabla y columna Oro (`COMMENT`) explicando qué significa cada métrica.

Criterio de aceptación: el modelo se evalúa con semanas que no usó para entrenar (validación temporal, no aleatoria) y el informe explica si supera o no a la línea base.

Fase 4: Clusters y orquestación en Databricks

El pipeline debe correr completo, sin intervención manual, en un job de Databricks.

1. **Desarrollo en cluster interactivo (all-purpose):** desarrollar y probar los notebooks en el cluster asignado. Documentar su configuración: versión de Databricks Runtime, tipo de nodo, número de workers, tiempo de autoterminación y librerías instaladas (OCR, MLflow).
2. **Separar el código en notebooks o archivos Python por capa:** `01_bronze`, `02_silver`, `03_gold`. Los valores que cambian (rutas, número de equipo, fechas, umbral de dirección) se reciben como parámetros (widgets), no escritos en el código.
3. **Crear un Job (Workflow) con 3 tareas encadenadas** Bronce → Plata → Oro, donde cada tarea depende de la anterior. La tarea Oro recalcula las variables y genera la predicción de la semana siguiente.
4. **Ejecutar el Job en un job cluster** (o en el cómputo que indique el catedrático) y no en el cluster interactivo. Explicar en el informe la diferencia de costo y propósito entre ambos tipos de cluster.
5. **Disparar el Job cuando haya fotos nuevas:** con un trigger por llegada de archivos al bucket Bronce o con una programación semanal (por ejemplo, cada martes). Configurar una notificación por correo si el job falla.
6. Adjuntar evidencia de al menos 2 ejecuciones exitosas del Job con fotos de semanas distintas: captura del historial de ejecuciones y duración de cada tarea.

Criterio de aceptación: el catedrático puede ejecutar el Job con "Run now" y las tablas Oro se actualizan sin tocar ningún notebook.

Fase 5: API en el Droplet de DigitalOcean

El resultado final se prueba consultando una API pública en el Droplet que lee los datos de la capa Oro.

1. **Acceso:** entrar al Droplet por SSH con las llaves asignadas (`ssh -i <llave> equipo-XX@<ip>`). Todo el trabajo del equipo vive en `/home/equipo-XX/`.

2. **Entorno aislado:** elegir uno de estos mecanismos y justificar la elección en el README:
   - **Docker** (recomendado): `Dockerfile` propio y, si aplica, `docker-compose.yml`.
   - **conda**: archivo `environment.yml`.
   - **uv**: `pyproject.toml` y `uv.lock`.
   - **venv**: `requirements.txt` con versiones fijas.
3. **Lectura de la capa Oro:** la API lee las tablas Oro desde S3 (Delta con `deltalake`/DuckDB/Polars, o Parquet) usando credenciales de solo lectura. Las credenciales van en un archivo `.env` en el servidor, nunca en el repositorio.
4. **Endpoints mínimos** (FastAPI o Flask):
   - `GET /health`: estado del servicio.
   - `GET /precios/actual`: último precio por combustible.
   - `GET /precios?combustible=diesel&desde=2026-10-01&hasta=2026-11-30`: histórico semanal con filtros.
   - `GET /prediccion?combustible=super`: predicción de la semana siguiente, por ejemplo `{"semana": "2026-11-10", "prediccion": "sube", "probabilidad": 0.68}`.
   - `GET /modelo/metricas`: exactitud del modelo y de la línea base.
   - Paginación o límite de resultados (`?limit=100`) y documentación accesible (Swagger en `/docs` si se usa FastAPI).
5. **Publicación:** la API escucha en el puerto `80XX` del equipo y queda corriendo aunque el equipo cierre su sesión SSH (contenedor con `restart: unless-stopped`, servicio `systemd` o similar).
6. **Prueba de usuario final:** incluir una colección de pruebas (`curl`, Postman o un script) que muestre el precio actual y la predicción para súper, regular y diésel.

Criterio de aceptación: desde una computadora externa, `curl http://<ip>:80XX/health` responde correctamente y cada endpoint devuelve datos de la capa Oro.

Entregables

Todos los entregables se suben a un repositorio de GitHub por equipo, llamado `equipo-XX-gasolina-gt`. El repositorio puede ser público o privado; si es privado, se da acceso al catedrático. Además, el Job debe quedar funcionando en Databricks y la API en línea en el Droplet.

En GitHub:

- [ ] **Diagrama de arquitectura de la plataforma** (`docs/`): buckets S3 por capa, workspace de Databricks con clusters y Job, Droplet con la API, y los flujos SSH y HTTP. Se entrega la imagen exportada y el archivo fuente editable (draw.io, Excalidraw o similar).
- [ ] **Código de los pipelines** (`pipelines/`): Bronce, Plata (extracción OCR y reglas de calidad), Oro (indicadores, variables, entrenamiento y predicción) y la definición exportada del Job. El código se sincroniza desde Databricks con Git folders.
- [ ] **Código de la API** (`api/`) con su entorno (Dockerfile, environment.yml, pyproject.toml o requirements.txt) y un `.env.example` sin valores reales.
- [ ] **Presentación para el jurado** (`presentacion/`): las diapositivas en PDF y el resumen de una página.
- [ ] `README.md` con la descripción del proyecto, las instrucciones para reproducir el pipeline y desplegar la API, y los enlaces al diagrama y a la presentación.
- [ ] Informe técnico corto (`docs/informe.pdf` , máximo 5 páginas): calidad de la extracción, variables del modelo, resultados contra la línea base, configuración de clusters y lecciones aprendidas.
- [ ] Historial de commits de todos los integrantes y un `.gitignore` que excluya `.env` , llaves SSH y credenciales.

Fuera de GitHub:

- [ ] Job de Databricks `equipo-XX-pipeline` , compartido con el catedrático, con su experimento de MLflow.
- [ ] API en línea en `http://<ip>:80XX` hasta la fecha de calificación.
- [ ] Presentación ante el jurado externo, con demo en vivo: ejecución del Job con fotos nuevas y consulta de la predicción en la API.

Estructura sugerida del repositorio:

```
equipo-XX-gasolina-gt/
├── README.md
├── .gitignore
├── docs/
│   ├── arquitectura.png        # diagrama de la plataforma
│   ├── arquitectura.drawio     # fuente editable (o .excalidraw)
│   └── informe.pdf
├── pipelines/
│   ├── 01_bronze.py            # ingesta de fotos
│   ├── 02_silver.py            # extracción OCR y reglas de calidad
│   ├── 03_gold.py              # indicadores, variables, modelo y predicción
│   └── job.json                # definición exportada del Job
├── api/
│   ├── app/main.py
```

```
|   ├── Dockerfile              # o environment.yml / pyproject.toml /
requirements.txt
|   └── .env.example            # variables sin valores reales
├── tests/
|   └── consultas_api.sh        # o colección de Postman
└── presentacion/
    ├── presentacion_jurado.pdf
    └── resumen_jurado.pdf
```

Rúbrica de evaluación

El proyecto vale 100 puntos; el mayor peso está en el pipeline medallón y en que la API funcione de punta a punta.

Criterio	Puntos	Qué se evalúa
Capa Bronce	10	Lectura de fotos por external location, carga incremental, columnas de auditoría
Capa Plata	15	Extracción de precios, estandarización, 3 reglas de calidad, cuarentena, exactitud medida a mano
Capa Oro	10	Precio semanal, variación, etiqueta de dirección, documentación de métricas
Modelo de predicción	10	Validación temporal, comparación con línea base, registro en MLflow
Clusters y Job	10	Cluster interactivo vs. job cluster, Job con 3 tareas, trigger o programación, alertas
API en el Droplet	20	Endpoints de precios y predicción, filtros, entorno aislado, servicio persistente, credenciales seguras
Repositorio y README	10	Repositorio en GitHub completo: diagrama, código de pipelines y API, presentación, reproducibilidad y commits de todos
Informe técnico	5	Justificación de decisiones y lecciones aprendidas
Presentación y demo	10	Calificado por el jurado externo
Total	**100**	

Puntos extra (hasta 5): el equipo cuya predicción de la Semana 5 acierte más direcciones reales (comparadas con las fotos de la semana siguiente) gana 3 puntos; los otros 2 se obtienen con pruebas automatizadas de la API, CI/CD hacia el Droplet o un dashboard en

Databricks SQL sobre la capa Oro.

Calendario e hitos

El proyecto se divide en 5 hitos semanales; cada uno se revisa en clase antes de pasar al siguiente.

1. **Semana 1:** acceso SSH al Droplet verificado y primeras fotos cargadas en `bronze.fotos_precios`.
2. **Semana 2:** extracción de precios funcionando y tabla `silver.precios` con reglas de calidad.
3. **Semana 3:** tablas Oro, primer modelo registrado en MLflow y Job de Databricks de punta a punta.
4. **Semana 4:** API desplegada en el Droplet con los endpoints de precios y predicción.
5. **Semana 5:** entrega del repositorio, informe y presentación final. Cada equipo deja registrada su predicción para la semana siguiente.

Presentación ante jurado

Cada equipo presenta ante un jurado de 2 o 3 personas ajenas a la clase. El jurado no conoce el curso ni el código, así que el equipo debe explicar primero el problema y su valor, y después la parte técnica.

Formato: 20 minutos por equipo. Son 10 de exposición, 5 de demo en vivo y 5 de preguntas del jurado.

Estructura sugerida:

1. **El problema (1 a 2 minutos):** por qué importa predecir el precio de la gasolina en Guatemala y para quién.
2. **Arquitectura:** el diagrama Bronce → Plata → Oro → API, explicado sin jerga innecesaria.
3. **De la foto al dato:** cómo extraen los precios y qué porcentaje aciertan según la revisión manual.
4. **El modelo:** qué predice, qué tan bien lo hace frente a la línea base y cuáles son sus limitaciones.
5. **Demo en vivo:** ejecutar el Job con fotos nuevas y consultar `/prediccion` en la API. El jurado puede hacer la consulta desde su propio teléfono con un código QR.
6. **Cierre:** lecciones aprendidas y qué harían con más tiempo o más datos.

Reglas para la presentación:

- Todos los integrantes participan, y el jurado puede preguntarle a cualquiera.

- 48 horas antes, el equipo entrega un resumen de una página para el jurado con el problema, la arquitectura, los resultados y la URL de la API.
- El equipo lleva un plan B por si falla la conexión: una grabación corta de la demo o capturas de pantalla.

Reglas, seguridad y buenas prácticas

Las credenciales y los recursos compartidos se tratan como en un entorno productivo real.

- **Llaves SSH:** son personales del equipo. No se comparten por chat, no se suben al repositorio y no se copian a otros servidores.
- **Credenciales de AWS:** nunca en notebooks ni en Git. En Databricks se usan las external locations configuradas; en el Droplet, un archivo `.env` con permisos `600`.
- **Recursos compartidos:** cada equipo trabaja solo en su prefijo de S3, sus esquemas de Databricks, su usuario y su puerto en el Droplet.
- **Costos:** el cluster interactivo debe tener autoterminación (máximo 30 minutos de inactividad). Clusters encendidos sin uso restan puntos.
- **Droplet:** no instalar paquetes a nivel de sistema ni reiniciar el servidor sin autorización del catedrático; usar el entorno aislado elegido.
- **Integridad académica:** se puede usar documentación, ejemplos y asistentes de IA, pero cada integrante debe poder explicar cualquier parte del código durante la presentación.

Fuentes

- AGN: precios de referencia del MEM vigentes del 30 de junio al 6 de julio de 2026
- La Hora: precios de referencia por región y modalidad
- Prensa Libre: precios de referencia con subsidio temporal